Programming II

Java IO

Michele Pasqua, PhD

UNIVERSITÀ
di VERONA
Dipartimento
di INFORMATICA

A.Y. 2025/2026

IO Streams

Input and output flow of data

In Java, IO operations rely on the stream abstraction (flow of data)

- IO streams are different from the `Stream` interface

An IO stream can be attached to

- A file on the disk
- Standard input, output, error // e.g., *System.in*
- A network connection
- A flow of data from/to hardware devices

IO operations work in the same way with all kinds of IO streams

Input and output flow of data (cont'd)

Provided by the `java.io` package

- Related exceptions are subclass of `IOException`

Byte-oriented streams

- Abstract classes `InputStream` and `OutputStream`
- Flow of bytes

Character-oriented streams

- Abstract classes `Reader` and `Writer`
- Flow of Unicode 16-bit characters

Byte stream

`InputStream` and `OutputStream` define byte-based input and output channels

- The constructor opens the stream
- The method `read()` gets one or more bytes from the stream // *InputStream*
- The method `write()` puts one or more bytes into the stream // *OutputStream*
- The method `close()` closes the stream

Concrete implementations are channel-specific

- Standard keyboard, screen, files, ...

Byte stream (cont'd)

Suitable for handling raw binary data such as images, audio, bytecode, ...

- `System.in` is a `BufferedInputStream`
- `System.out` is a `BufferedOutputStream`

Concrete implementations

`BufferedInputStream`	Read data efficiently with buffering
`DataInputStream`	Read Java primitive data types
`FileInputStream`	Read from a file
`BufferedOutputStream`	Write data efficiently with buffering
`DataOutputStream`	Write Java primitive data types
`FileOutputStream`	Write to a file

Binary file IO

```
public static void main(String args[]) throws IOException {
  FileInputStream fis = null;
  FileOutputStream fos = null;
  try {
    fis = new FileInputStream(args[0]);
    fos = new FileOutputStream(args[1]);
    int b;
    while ((b = fis.read()) != -1) {     // throws IOException
      fos.write(b);                      // throws IOException
    }
  } catch (FileNotFoundException fnfe) {
    System.out.println("File not found...");
  } finally {
    if (fis != null) fis.close();        // throws IOException
    if (fos != null) fos.close();        // throws IOException
  }
}
```

Character stream

`Reader` and `Writer` define character-based input and output channels

- The constructor opens the stream
- The method `read()` gets one or more characters from the stream // *Reader*
- The method `write()` puts one or more characters into the stream // *Writer*
- The method `close()` closes the stream

Concrete implementations are channel-specific

- Standard keyboard, screen, files, ...

Character stream (cont'd)

Suitable for handling text data (typically files or strings)

Concrete implementations

`BufferedReader`	Read text efficiently with buffering
`InputStreamReader`	Wrapper for reading from a byte-based channel
`StringReader`	Wrapper for reading from a string
`FileReader`	Read from a text file
`BufferedWriter`	Write text efficiently with buffering
`OutputStreamWriter`	Wrapper for writing to a byte-based channel
`StringWriter`	Wrapper for writing to a string
`FileWriter`	Write to a text file

Text file IO

```
public static void main(String args[]) throws IOException {
  BufferedReader br = null;
  BufferedWriter bw = null;
  try {
    br = new BufferedReader(new FileReader("i.txt"));
    bw = new BufferedWriter(new FileWriter("o.txt"));
    String line;
    while ((line = br.readLine()) != null) {    // throws IOException
      bw.write(line);                           // throws IOException
    }
  } catch (FileNotFoundException fnfe) {
    System.out.println("File not found...");
  } finally {
    if (br != null) br.close();                 // throws IOException
    if (bw != null) bw.close();                 // throws IOException
  }
}
```

Text file IO (cont'd)

From Java 7, we can use try-with-resources

- Streams are automatically closed

```
public static void main(String args[]) {
  try (BufferedReader br = new BufferedReader(new FileReader("i.txt"));
       BufferedWriter bw = new BufferedWriter(new FileWriter("o.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {     // throws IOException
      bw.write(line);                            // throws IOException
    }
  } catch (FileNotFoundException fnfe) {
    System.out.println("File not found...");
  } catch (IOException ioe) {
    System.out.println("Error while reading/writing the file...");
  }
}
```

Files and URLs

Files

The `java.io` package provides the class `File`

- Modeling pathnames, like files and directories
- Absolute or relative paths

Provides methods to deal with files and directories

- `create()`
- `delete()`
- `exists()`
- `renameTo()`
- `getParent()`
- `mkdir()`
- `getAbsolutePath()`
- `isFile()`
- `isDirectory()`
- `listFiles()`

List files

List the files contained in the current directory

```
File cwd = new File(".");
for (File f : cwd.listFiles()) { // we should check if it is a directory
  System.out.println(f.getName() + "␣" + f.length());
}
```

Absolute and relative path

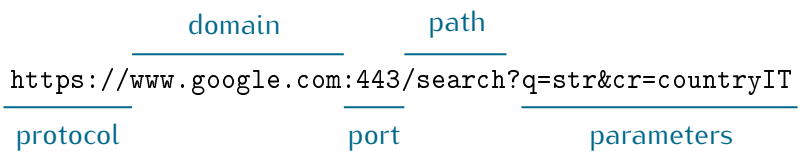
```
File source = new File("Main.java");
// relative path
System.out.println(source.getName()); // 'Main.java'
// absolute path
System.out.println(source.getAbsolutePath()); // '/path/to/Main.java'
```

URLs

The `java.net` package provides the class `URL`

- Modeling a Uniform Resource Locator (URL)
- Essentially, a pointer to some resource on the web

URL components



Download a file

`URL` constructor is now deprecated, use `URI.toURL()` instead

- `URL www = new URI("http://info.cern.ch/index.html").toURL();`

Download and print on screen a HTML page

```
try {
  URL www = new URI("http://info.cern.ch/index.html").toURL();
  BufferedInputStream in = new BufferedInputStream(www.openStream());
  byte[] dataBuffer = new byte[1024];
  while (in.read(dataBuffer, 0, 1024) != -1) {
    System.out.println(new String(dataBuffer));
  }
} catch (IOException | URISyntaxException e) throw new RuntimeException(e);
```

Craft URLs

Alternative constructor of the `URI` class

```
String protocol = "http";                                                    1
String host = "www.google.com";                                              2
String path = "/search";                                                     3
String params = "q=str\&cr=countryIT";                                       4
                                                                             5
URL url = new URI(protocol, null, host, -1, path, params, null).toURL();     6
// 'http://www.google.com/search?q=str\&cr=countryIT'                         7
System.out.println(url.toString());                                          8
```

Structured Data

CSV files

A CSV (comma-separated values) file is a text file representing tabular data

- Like spreadsheet or database content
- Standard format, easy to parse

Each file contains a series of lines representing the rows of a table

- The columns of the table are separated with commas
- No data types, all values in the files are strings
- Escaping is needed when values contain commas or quotes

CSV in Java

No native support, a third party library is needed

- For instance, OpenCSV

```
<dependency>          <!-- to add to the pom.xml Maven configuration file -->
  <groupId>com.opencsv</groupId>
  <artifactId>opencsv</artifactId>
  <version>5.12.0</version>
</dependency>
```

The library provides classes `CSVReader` and `CSVWriter` to read and write CSV files

OpenCSV

Read a CSV file

```
// From a File object, with the default reader
File csvFile = new File("i.csv");
CSVReader csvReader = new CSVReader(new FileReader(csvFile));

// From a filename, with a custom reader
CSVReader csvReader = new CSVReaderBuilder(new FileReader("i.csv"))
                          .withSkipLines(1)
                          .withSeparator(';')
                          .build();

// Read CSV line by line
List<String[]> table = new ArrayList<>();
for (String[] row; (row = csvReader.readNext()) != null; )  // throws CsvException
  table.add(row);

// Read the whole CSV at once
List<String[]> table = csvReader.readAll();                 // throws CsvException
```

OpenCSV (cont'd)

Write a CSV file

```
// To a File object, with the default writer
File csvFile = new File("o.csv");
CSVWriter csvWriter = new CSVWriter(new FileWriter(csvFile));

// To a filename, with a custom writer
CSVWriter csvWriter = new CSVWriter(new FileWriter("o.csv"), ';');

// Write CSV line by line
for (String[] row : table)
  csvWriter.writeNext(row);
csvWriter.flush();           // not needed if we close the file

// Write the whole CSV at once
csvWriter.writeAll(table);
csvWriter.flush();           // not needed if we close the file
```

JSON files

A JSON (JavaScript Object Notation) file is a text file representing a human-readable representation of structured data

- Standard format, easy to parse
- Universally used on the web

Each file contains either *// recursively*

- Objects: key-value pairs delimited by curly brackets `{ }`
- Arrays: ordered lists of values delimited by square brackets `[ ]`

Values can be strings, numbers, booleans, objects or arrays

JSON in Java

No native support, a third party library is needed

- For instance, Gson from Google

```
<dependency>          <!-- to add to the pom.xml Maven configuration file -->
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.13.2</version>
</dependency>
```

The library provides the class `Gson` with the methods `fromJson` and `toJson` to parse and build JSON content

- The JSON content can be read from or written to file
- Binding with user-defined classes simplify the process

Gson

Build JSON content from a class instance

```
Gson gson = new Gson();
Student sam = new Student("Sam", 123);
String json = gson.toJson(sam);
System.out.println(json); // prints '{"matricola":123,"name":"Sam"}'
```

Parse JSON content into a class instance

```
Gson gson = new Gson();
String json = "{\"matricola\":123,\"name\":\"Sam\"}";
Student sam = gson.fromJson(json, Student.class); // throws JsonSyntaxException
System.out.println(sam.toString()); // prints 'Sam 123'
```

Gson (cont'd)

Write to JSON file

```
try (FileWriter writer = new FileWriter("sam.json")) {
  Gson gson = GsonBuilder().setPrettyPrinting().create();
  Student sam = new Student("Sam", 123);
  gson.toJson(sam, writer); // throws JsonIOException
} catch (IOException ioe) {
  throw new RuntimeException(ioe);
}
```

Read from JSON file

```
try (FileReader reader = new FileReader("sam.json")) {
  Gson gson = new Gson();
  Student sam = gson.fromJson(reader, Student.class); // throws JsonSyntaxException, JsonIOException
} catch (IOException ioe) {
  throw new RuntimeException(ioe);
}
```

Q + A